

TEMA 1. ANÁLISIS DE RENDIMIENTO CON SQL PROFILER Y EXTENDED EVENTS

Proyecto 1: Respaldo Completo Institucional de la Base de Datos GradosTitulos

1. Enunciado del ejercicio

La Oficina de Tecnologías de Información de la universidad ha detectado que el módulo de **registro de expedientes de bachiller y título profesional** presenta lentitud durante las horas de mayor demanda. Se solicita implementar una sesión de monitoreo utilizando **Extended Events** para capturar todas las consultas cuya duración sea superior a **1000 milisegundos** sobre la base de datos **GradosTitulos**. El estudiante deberá identificar las consultas más costosas ejecutadas sobre los esquemas:

- Academico
- Tramite
- Evaluacion
- Documento

El sistema deberá registrar automáticamente las consultas más costosas ejecutadas por los usuarios durante la jornada laboral y almacenar la información en una tabla histórica para posteriores auditorías de desempeño. El estudiante deberá diseñar un mecanismo que permita:

- Iniciar la sesión de monitoreo;
- Registrar las consultas ejecutadas sobre la base de datos GradosTitulos;
- Importar periódicamente la información del archivo de eventos a una tabla de auditoría;
- Generar un reporte de las consultas con mayor duración, mayor consumo de CPU y mayor número de lecturas lógicas. Al finalizar, deberá generar un reporte con las consultas capturadas.

2. Script de la solución en T-SQL

```

-- =====
-- TEMA 1 - PROYECTO 1: Monitoreo de Consultas con Extended Events (XEvents)
-- Base de Datos: GradosTitulos
-- =====

USE GradosTitulos;
GO

-- -----
-- PASO 1: Creación de la Sesión de Eventos Extendidos
-- -----

IF EXISTS (SELECT * FROM sys.server_event_sessions WHERE name = 'XEvent_Monitor_GradosTitulos')
BEGIN
    DROP SERVER EVENT SESSION XEvent_Monitor_GradosTitulos ON SERVER;
END
GO

CREATE SERVER EVENT SESSION [XEvent_Monitor_GradosTitulos] ON SERVER
ADD EVENT sqlserver.sql_statement_completed(
    ACTION(
        sqlserver.database_name,
        sqlserver.nt_username,
        sqlserver.sql_text,
        sqlserver.session_id
    )
    WHERE (
        [sqlserver].[database_name] = N'GradosTitulos'
        AND [duration] >= 1000000 -- Expresado en microsegundos (1,000,000 microsegundos = 1 segundo)
        AND (
            [sqlserver].[like_i_sql_text]([sqlserver].[sql_text], N'%Academico.%') OR
            [sqlserver].[like_i_sql_text]([sqlserver].[sql_text], N'%Tramite.%') OR
            [sqlserver].[like_i_sql_text]([sqlserver].[sql_text], N'%Evaluacion.%') OR
            [sqlserver].[like_i_sql_text]([sqlserver].[sql_text], N'%Documento.%')
        )
    )
)
)
)

```

3. Justificación técnica de la solución aplicada

- **Uso de Extended Events frente a SQL Profiler:** Se seleccionó de manera nativa la arquitectura de *Extended Events* ya que consume drásticamente menos recursos del servidor, operando en el motor de base de datos de manera ligera, ideal para entornos de producción con alta demanda transaccional.
- **Filtros de Duración en Microsegundos:** El predicado `[duration] >= 1000000` se define bajo la escala de microsegundos, el estándar del motor de SQL Server para el evento `sql_statement_completed`, asegurando así aislar las peticiones con lentitud real de ejecución (superiores a 1 segundo).
- **Filtros por Esquema Mediante Predicados:** Se incluyeron cláusulas lógicas condicionales (`like_i_sql_text`) directamente en la sesión para acotar el análisis únicamente a cadenas que involucren los esquemas solicitados (Academico, Tramite, Evaluacion, Documento). Esto previene el crecimiento innecesario del archivo físico en disco.

- **Procesamiento de Archivos .xel vía XML:** Dado que el target event_file guarda la metadata de forma binaria, se empleó la función del sistema sys.fn_xe_file_target_read_file transformando la información en un tipo de dato XML, lo cual permite usar métodos de tipado nativo de XQuery (.value) para estructurar datos limpios y mapeables a columnas relacionales relativas a CPU, lecturas lógicas y duración.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso Excluyente de Mayúsculas en Palabras Clave:** Todas las instrucciones de control del sistema y del estándar T-SQL (CREATE, SELECT, WHERE, INSERT INTO) han sido estructuradas en mayúsculas fijas para optimizar su interpretación por el parser SQL y estandarizar la legibilidad del código.
- **Garantía de Idempotencia:** El script incluye bloques de verificación condicional previos (IF EXISTS o IF NOT EXISTS), lo que faculta ejecutar el archivo .sql múltiples veces sin provocar fallas por duplicidad de objetos en el servidor.
- **Configuración del Target event_file con Rollover:** Se limitó el tamaño de los archivos a un volumen fijo de 5 MB y un máximo de 4 archivos secuenciales. Esto mitiga el riesgo de saturar el almacenamiento del servidor de bases de datos ante picos de concurrencia.
- **Prevención de Duplicados en Procesos ETL Históricos:** Al transferir datos del archivo plano .xel a la base de datos se aplicó un filtro WHERE NOT EXISTS, asegurando la consistencia e integridad de las auditorías históricas sin generar redundancia analítica.

Proyecto 2: Implementación de Backups Diferenciales para Trámites Académicos

1. Enunciado del ejercicio

Durante los procesos de matrícula para obtención del título profesional, varios usuarios reportan tiempos elevados en la generación de resoluciones. La Dirección de Informática solicita construir una solución de monitoreo que capture automáticamente:

- Consultas lentas.
- Deadlocks.
- Bloqueos.
- Errores superiores al nivel 16.
- Uso excesivo de CPU.

Toda la información deberá almacenarse en un archivo de eventos para su posterior análisis.

2. Script de la solución en T-SQL

```

-- TEMA 1 - PROYECTO 2: Sesión Avanzada de Extended Events para Diagnóstico
-- Base de Datos: GradosTitulos
-- =====

USE GradosTitulos;
GO

-- -----
-- PASO 1: Eliminación previa de la sesión si ya existe (Garantía de Idempotencia)
-- -----
IF EXISTS (SELECT * FROM sys.server_event_sessions WHERE name = 'XEvent_Diagnostico_I
BEGIN
    DROP SERVER EVENT SESSION XEvent_Diagnostico_Matriculas ON SERVER;
END
GO

-- -----
-- PASO 2: Creación de la Sesión de Eventos Extendidos
-- -----
CREATE SERVER EVENT SESSION [XEvent_Diagnostico_Matriculas] ON SERVER
-- 1. Captura de Consultas Lentas y Uso Excesivo de CPU
ADD EVENT sqlserver.sql_statement_completed(
    ACTION(
        sqlserver.database_name,
        sqlserver.nt_username,
        sqlserver.sql_text,
        sqlserver.session_id,
        sqlserver.client_app_name
    )
    WHERE (
        [sqlserver].[database_name] = N'GradosTitulos'
        AND (
            [duration] >= 2000000 -- Duración mayor o igual a 2 segundos (en microseg)
            OR [cpu_time] >= 1000000 -- Consumo de CPU mayor o igual a 1 segundo (en
        )
    )
),

```

3. Justificación técnica de la solución aplicada

- **Monitoreo Multievento Centralizado:** En lugar de crear múltiples herramientas de rastreo individuales, se diseñó una única sesión de *Extended Events* que consolida cuatro motores de diagnóstico críticos (*sql_statement_completed*, *xml_deadlock_report*, *blocked_process_report* y *error_reported*), garantizando una visión holística del problema de rendimiento en las resoluciones.
- **Correlación de Eventos mediante Causality Tracking:** Se activó la propiedad `TRACK_CAUSALITY = ON`. Esto adjunta un identificador único secuencial a las acciones del servidor, permitiendo analizar si un error de severidad alta o un bloqueo prolongado fue la causa directa de una consulta lenta específica.
- **Activación del Umbral de Bloqueos (*blocked process threshold*):** El evento *blocked_process_report* requiere de manera obligatoria que el servidor tenga configurado un límite de tiempo. Se estableció en **5 segundos** mediante `sp_configure`, permitiendo que el motor

genere un reporte automático en XML cada vez que una transacción de matrícula detenga a otra por un periodo perjudicial.

- **Aislamiento de Errores Críticos:** Al filtrar [severity] > 16, se descartan las advertencias menores o errores informativos de sintaxis introducidos por el usuario, enfocando el almacenamiento únicamente en fallas severas de base de datos, problemas de hardware o transacciones abortadas por el sistema.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Preservación de Recursos del Sistema:** La sesión utiliza ALLOW_MULTIPLE_EVENT_LOSS en su modo de retención. Esto prioriza la estabilidad y rendimiento del servidor de producción ante ráfagas extremas de transacciones de matrícula, prefiriendo omitir algún evento en memoria antes que ralentizar las operaciones de los usuarios.
- **Estandarización Estricta de la Sintaxis T-SQL:** Al igual que en los estándares de ingeniería de software corporativos, todas las sentencias de control, funciones del sistema y comandos de definición están estrictamente tipados en mayúsculas (CREATE SERVER EVENT SESSION, ALTER SERVER, WHERE, AND), manteniendo uniformidad visual y profesionalismo en el entregable.
- **Control del Ciclo de Vida del Almacenamiento (Target):** Al limitar el tamaño del archivo a 10 MB con un máximo de 5 archivos circulares (*rollover*), se previene de forma absoluta el desbordamiento de espacio en los discos del servidor local, reciclando los registros más antiguos automáticamente.

Proyecto 3: Estrategia Integral Full + Differential + Log Backup

1. Enunciado del ejercicio

La universidad desea implementar un sistema permanente de diagnóstico del rendimiento para la base de datos GradosTitulos. El sistema deberá registrar automáticamente las consultas más costosas ejecutadas por los usuarios durante la jornada laboral y almacenar la información en una tabla histórica para posteriores auditorías de desempeño.

(Nota: De acuerdo con la secuencia lógica del documento técnico integrado, este módulo consolida la automatización de la recolección periódica del rendimiento analizado en los proyectos anteriores mediante la programación de scripts robustos de mantenimiento y extracción).

2. Script de la solución en T-SQL

```

-- =====
-- TEMA 1 - PROYECTO 3: Automatización de Diagnóstico y Extracción de Rendimiento
-- Base de Datos: GradosTitulos
-- =====

USE GradosTitulos;
GO

-- -----
-- PASO 1: Procedimiento Almacenado para la Extracción Automatizada de Datos
-- -----

IF EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[USP_Actualizar_Auditoria_Rendimiento]'))
BEGIN
    DROP PROCEDURE [dbo].[USP_Actualizar_Auditoria_Rendimiento];
END
GO

CREATE PROCEDURE [dbo].[USP_Actualizar_Auditoria_Rendimiento]
AS
BEGIN
    SET NOCOUNT ON;

    -- Verificar que la tabla histórica exista antes de la inserción
    IF EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[Historico_Consultas_Lentas]'))
    BEGIN
        -- Insertar registros nuevos procesando el archivo XML de Extended Events
        INSERT INTO [dbo].[Historico_Consultas_Lentas] (
            [FechaEvento], [SesionId], [Usuario], [BaseDatos], [ConsultaSQL], [Duracion_ms], [CPU_ms], [LecturasLogicas], [Fragmento_Consulta]
        )
        SELECT
            event_data.value('(event/@timestamp)[1]', 'DATETIMEOFFSET') AS FechaEvento,
            event_data.value('(event/action[@name="session_id"]/value)[1]', 'INT') AS SesionId,
            event_data.value('(event/action[@name="nt_username"]/value)[1]', 'NVARCHAR(128)') AS Usuario,
            event_data.value('(event/action[@name="database_name"]/value)[1]', 'NVARCHAR(128)') AS BaseDatos,
            event_data.value('(event/action[@name="sql_text"]/value)[1]', 'NVARCHAR(4096)') AS ConsultaSQL,
            event_data.value('(event/action[@name="duration"]/value)[1]', 'INT') AS Duracion_ms,
            event_data.value('(event/action[@name="cpu_time"]/value)[1]', 'INT') AS CPU_ms,
            event_data.value('(event/action[@name="logical_reads"]/value)[1]', 'INT') AS LecturasLogicas,
            event_data.value('(event/action[@name="sql_text_substring"]/value)[1]', 'NVARCHAR(200)') AS Fragmento_Consulta
    
```

```

-- -----
-- PASO 2: Creación de la Vista para Reportes de Consumo de la Jornada Laboral
-- -----

IF EXISTS (SELECT * FROM sys.views WHERE object_id = OBJECT_ID(N'[dbo].[VW_Reporte_Consumo_Critico]'))
BEGIN
    DROP VIEW [dbo].[VW_Reporte_Consumo_Critico];
END
GO

CREATE VIEW [dbo].[VW_Reporte_Consumo_Critico]
AS
SELECT TOP 100 PERCENT
    CAST([FechaEvento] AS DATETIME) AS [Fecha_Hora],
    [Usuario] AS [Usuario_Ejecutor],
    [Duracion_ms] AS [Duracion_MS],
    [CPU_ms] AS [CPU_MS],
    [LecturasLogicas] AS [Lecturas_Logicas],
    SUBSTRING([ConsultaSQL], 1, 200) AS [Fragmento_Consulta]
FROM [dbo].[Historico_Consultas_Lentas]
WHERE CAST([FechaEvento] AS TIME) BETWEEN '08:00:00' AND '18:00:00' -- Rango de la jornada laboral
GO

-- -----
-- PASO 3: Ejecución de Prueba y Generación del Reporte Consolidado
-- -----

-- 1. Ejecutar el procedimiento para sincronizar los datos del archivo a la tabla histórica
EXEC [dbo].[USP_Actualizar_Auditoria_Rendimiento];
GO

-- 2. Consultar la vista ordenada por las métricas de mayor consumo de recursos
SELECT
    [Fecha_Hora],
    [Usuario_Ejecutor],
    [Duracion_MS],
    [CPU_MS],
    [Lecturas_Logicas],
    [Fragmento_Consulta]
FROM [dbo].[VW_Reporte_Consumo_Critico]
ORDER BY [Duracion_MS] DESC, [CPU_MS] DESC, [Lecturas_Logicas] DESC;
GO

```

3. Justificación técnica de la solución aplicada

- **Encapsulamiento Lógico Mediante Stored Procedures:** La lógica ETL de lectura e inserción del archivo binario .xel se delegó a un procedimiento almacenado (USP_Actualizar_Auditoria_Rendimiento). Esto permite aislar el código transaccional, reducir la latencia de red y posibilitar su invocación automática mediante el Agente de SQL Server (SQL Server Agent).
- **Uso de SET NOCOUNT ON:** Se añadió al inicio del procedimiento para suprimir el envío de mensajes (1 row(s) affected) al cliente por cada fila procesada. Esto optimiza el rendimiento general del motor y reduce el tráfico de red.
- **Segmentación Temporal de la Jornada Laboral:** La vista VW_Reporte_Consumo_Critico implementa un filtro de rango horario explícito (BETWEEN '08:00:00' AND '18:00:00'). De este modo se aísla el ruido analítico causado por tareas programadas nocturnas (como reindexaciones o backups de mantenimiento) y se enfoca el diagnóstico puramente en el horario de atención al estudiante.
- **Control de Errores con RAISERROR:** Permite que el script sea autodiagnóstico, levantando una alerta formal de severidad 16 en caso de inconsistencia estructural en los objetos relacionales de auditoría.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso de Esquemas Explícitos y Nombres Calificados:** Cada objeto referenciado utiliza su prefijo de esquema correspondiente ([dbo].[Tabla]), lo que evita penalizaciones en la caché del plan de ejecución del motor de SQL Server al buscar los objetos en el catálogo.
- **Protección contra Truncamientos en Reportes:** En la definición de la vista se utilizó la función SUBSTRING([ConsultaSQL], 1, 200) para los reportes de visualización rápida. Esto garantiza que las pantallas de monitoreo carguen ágilmente sin saturar la memoria del administrador con textos de consultas masivas.
- **Garantía de Idempotencia Estricta:** El uso consistente de bloques de comprobación de preexistencia garantiza que la solución pueda ser desplegada en entornos de desarrollo, pruebas o producción de forma transparente sin generar colisiones ni errores de compilación.

TEMA 2. ESTADÍSTICAS E ÍNDICES

Proyecto 1: Optimización de índices para consultas de estudiantes

1. Enunciado del ejercicio

Las consultas realizadas sobre las tablas **Academico.Persona**, **Academico.Matricula** y **Tramite.Tramite** presentan tiempos elevados durante la búsqueda de estudiantes. Se solicita desarrollar un conjunto de scripts que permitan:

- Analizar índices existentes.
- Detectar índices faltantes.
- Crear índices Clustered y NonClustered.
- Crear índices INCLUDE.
- Comparar tiempos antes y después de la optimización.

El resultado deberá demostrar la mejora del rendimiento utilizando estadísticas de ejecución.

2. Script de la solución en T-SQL

```
-- =====
-- TEMA 2 - PROYECTO 1: Optimización de Índices y Análisis de Rendimiento
-- Base de Datos: GradosTitulos
-- =====

USE GradosTitulos;
GO

-- -----
-- PASO 1: Analizar Índices Existentes en las Tablas Objetivos
-- -----

SELECT
    OBJECT_SCHEMA_NAME(i.object_id) AS [Esquema],
    OBJECT_NAME(i.object_id) AS [Tabla],
    i.name AS [Nombre_Indice],
    i.type_desc AS [Tipo_Indice]
FROM sys.indexes i
WHERE i.object_id IN (
    OBJECT_ID('Academico.Persona'),
    OBJECT_ID('Academico.Matricula'),
    OBJECT_ID('Tramite.Tramite')
);
GO

-- -----
-- PASO 2: Detectar Índices Faltantes Sugeridos por el Motor (DMVs)
-- -----

SELECT
    CONVERT(DECIMAL(18,2), migs.user_seeks * migs.avg_total_user_cost * (migs.avg_usi
mid.statement AS [Tabla_Destino],
    CONVERT(NVARCHAR(MAX), mid.equality_columns) AS [Columnas_Igualdad],
    CONVERT(NVARCHAR(MAX), mid.inequality_columns) AS [Columnas_Desigualdad],
    CONVERT(NVARCHAR(MAX), mid.included_columns) AS [Columnas_Include]
FROM sys.dm_db_missing_index_groups mig
INNER JOIN sys.dm_db_missing_index_group_stats migs ON migs.group_handle = mig.index
INNER JOIN sys.dm_db_missing_index_details mid ON mid.index_handle = mig.index_handl
WHERE mid.database_id = DB_ID()
    AND mid.object_id IN (OBJECT_ID('Academico.Persona'), OBJECT_ID('Academico.Matricu
ORDER BY [Mejora_Estimada] DESC;
GO
```

3. Justificación técnica de la solución aplicada

- **Análisis Dinámico por DMVs:** Se utilizaron las Vistas de Gestión Dinámica (`sys.dm_db_missing_index_*`) para calcular de forma matemática el impacto potencial de la indexación. Esto asegura que los índices no se creen al azar, sino fundamentados en las deficiencias de acceso del motor.
- **Mecanismo de Índices de Cobertura (INCLUDE):** Al estructurar `IX_Academico_Persona_Apellido`, se definió `Apellido` como la clave del índice porque se encuentra en el predicado de filtrado (`WHERE`), mientras que `Nombre` se añade en la cláusula

INCLUDE. Esto evita que SQL Server realice un salto costoso a la tabla base (*Key Lookup*), resolviendo la consulta directamente desde las páginas de datos del índice.

- **Optimización de Joins Intermedios:** El índice compuesto sobre `Academico.Matricula(IdPersona, IdMatricula)` acelera el cruce relacional (*Hash Match* o *Merge Join*) de las claves foráneas, reduciendo drásticamente las lecturas lógicas requeridas en memoria.
- **Uso de SET STATISTICS:** Las opciones IO y TIME obligan al SSMS a imprimir de manera detallada el número de lecturas lógicas (páginas leídas en memoria caché) y el tiempo exacto de CPU, proporcionando la evidencia cuantitativa de mejora requerida en el enunciado.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso de Prefijos Normalizados de Nomenclatura:** Se utilizó el estándar de nomenclatura empresarial (CX_ para índices *Clustered*, e IX_ para índices *NonClustered*), garantizando la fácil identificación de los elementos del esquema por otros administradores de bases de datos.
- **Previsión de Eliminación para Despliegues Seguros:** Cada sentencia de creación viene precedida por una validación de existencia (IF EXISTS), evitando errores de ejecución si los índices ya habían sido creados en fases de pruebas previas.
- **Inclusión de Columnas Clave Estrictas:** Se minimizó el uso de columnas dentro de la sección INCLUDE para no aumentar desmedidamente el tamaño físico del archivo de base de datos (.mdf) ni penalizar las operaciones de inserción, actualización y borrado (INSERT, UPDATE, DELETE).

Proyecto 2: Detección y corrección de fragmentación

1. Enunciado del ejercicio

Después de varios meses de operaciones se observa disminución del rendimiento debido al crecimiento de las tablas. Se solicita desarrollar un procedimiento almacenado que:

- Detecte fragmentación mayor al 20%.
- Reorganice índices cuando la fragmentación sea moderada.
- Reconstruya índices cuando supere el 40%.
- Actualice estadísticas automáticamente.
- Genere un reporte con el porcentaje de mejora obtenido.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- -----
-- PASO 1: Creación de la Tabla Histórica para el Reporte de Mejoras
-- -----

IF NOT EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[Log_Mantenimiento_Indices]'))
BEGIN
    CREATE TABLE [dbo].[Log_Mantenimiento_Indices] (
        [IdLog] INT IDENTITY(1,1) PRIMARY KEY,
        [Esquema] NVARCHAR(128),
        [Tabla] NVARCHAR(128),
        [Indice] NVARCHAR(128),
        [Fragmentacion_Inicial] DECIMAL(5,2),
        [Accion_Realizada] NVARCHAR(50),
        [FechaEjecucion] DATETIME DEFAULT GETDATE()
    );
END
GO

-- -----
-- PASO 2: Creación del Procedimiento Almacenado de Mantenimiento Integral
-- -----

IF EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[USP_Mantenimiento_Fragmentacion_Indices]'))
BEGIN
    DROP PROCEDURE [dbo].[USP_Mantenimiento_Fragmentacion_Indices];
END
GO

CREATE PROCEDURE [dbo].[USP_Mantenimiento_Fragmentacion_Indices]
AS
BEGIN
    SET NOCOUNT ON;

    -- Declaración de variables para el control del cursor
    DECLARE @Esquema NVARCHAR(128);
    DECLARE @Tabla NVARCHAR(128);

```

3. Justificación técnica de la solución aplicada

- **Uso de sys.dm_db_index_physical_stats:** Esta función de gestión dinámica lee la estructura física del árbol B de los índices en tiempo real. Se configuró con el parámetro 'LIMITED', que es la mejor práctica de rendimiento, ya que solo escanea las páginas de nivel superior (padre) y raíz sin leer de forma exhaustiva las páginas de datos hoja, evitando sobrecargar la memoria.
- **Discriminación Dinámica de Umbrales:** Se implementó una lógica de bifurcación estricta controlada por variables (IF...ELSE). Al detectar fragmentaciones entre 20% y 40% se aplica REORGANIZE (proceso en línea, ligero y que consume pocos recursos de transacciones). Si supera el 40%, se aplica de forma mandatoria un REBUILD, el cual reconstruye por completo el índice físico, eliminando el espacio muerto en las páginas.
- **Estrategia Complementaria con UPDATE STATISTICS ... WITH FULLSCAN:** Al reorganizar o reconstruir un índice, las estadísticas no siempre quedan actualizadas con la máxima precisión.

Forzar un WITH FULLSCAN asegura que el optimizador de consultas tenga la distribución exacta de los datos para generar planes de ejecución óptimos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Manejo de SQL Dinámico de Forma Segura:** El uso de `sp_executesql` garantiza el correcto aislamiento sintáctico de las variables de cadena y encapsula de forma segura los nombres de los esquemas, tablas e índices del sistema universitario.
- **Control de Excepciones y Resiliencia (TRY...CATCH):** Se incluyó un bloque de captura de errores para manejar escenarios donde la opción `ONLINE = ON` de reconstrucción de índices no esté soportada (por ejemplo, en ediciones SQL Server Standard o si el índice contiene tipos de datos LOB complejos), garantizando que el mantenimiento continúe sin detener el script abruptamente.
- **Cierre Eficiente de Cursores:** Se asegura la ejecución imperativa de los comandos `CLOSE` y `DEALLOCATE` del cursor `curIndíces` para liberar inmediatamente los bloqueos internos del catálogo del sistema y evitar fugas de memoria en el servidor.

Proyecto 3: Automatización del mantenimiento de índices

1. Enunciado del ejercicio

La Oficina de Tecnologías de la Información requiere automatizar el mantenimiento de índices para que se ejecute todos los fines de semana a la medianoche. Se solicita diseñar una solución integral que:

- Consolide los procesos de evaluación de fragmentación y estadísticas.
- Registre las métricas previas y posteriores de rendimiento.
- Genere alertas o reportes del estado final de salud de los índices de la base de datos **GradosTítulos**.

(Nota: En la secuencia de diseño curricular y de laboratorios, este proyecto actúa como el cierre operativo del Tema 2, orquestando y empaquetando la lógica de los scripts anteriores en un entorno de producción automatizado).

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- -----
-- PASO 1: Creación de la Vista de Salud de Índices para Reportes Dominicales
-- -----

IF EXISTS (SELECT * FROM sys.views WHERE object_id = OBJECT_ID(N'[dbo].[VW_Salud_Indi
BEGIN
    DROP VIEW [dbo].[VW_Salud_Indices_Consolidado];
END
GO

CREATE VIEW [dbo].[VW_Salud_Indices_Consolidado]
AS
SELECT
    OBJECT_SCHEMA_NAME(ps.object_id) AS [Esquema],
    OBJECT_NAME(ps.object_id) AS [Tabla],
    i.name AS [Indice],
    ps.index_type_desc AS [Tipo_Indice],
    CAST(ps.avg_fragmentation_in_percent AS DECIMAL(5,2)) AS [Fragmentacion_Porcenta
    ps.page_count AS [Total_Paginas],
    CASE
        WHEN ps.avg_fragmentation_in_percent <= 5.0 THEN 'Excelente'
        WHEN ps.avg_fragmentation_in_percent > 5.0 AND ps.avg_fragmentation_in_perce
        WHEN ps.avg_fragmentation_in_percent > 20.0 AND ps.avg_fragmentation_in_perce
        ELSE 'Requiere Reconstruir'
    END AS [Estado_Salud]
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ps
INNER JOIN sys.indexes i ON ps.object_id = i.object_id AND ps.index_id = i.index_id
WHERE i.name IS NOT NULL AND ps.index_id > 0;
GO

-- -----
-- PASO 2: Procedimiento Maestro de Evaluación y Notificación
-- -----

IF EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[USP_Ejecuti
BEGIN

```

3. Justificación técnica de la solución aplicada

- **Abstracción Analítica mediante Vistas de Sistema:** Se estructuró la vista `VW_Salud_Indices_Consolidado` para centralizar las reglas de negocio técnicas del servidor. Al categorizar los rangos de fragmentación (Excelente, Saludable, Requiere Reorganizar, Requiere Reconstruir), se transforma la data binaria de las páginas en indicadores clave de rendimiento (KPIs) legibles por la OTI.
- **Orquestación Modular Basada en Reusabilidad:** El procedimiento maestro `USP_Ejecutar_Mantenimiento_Programado` implementa una arquitectura modular. Invoca de manera nativa los subcomponentes lógicos ya construidos en fases anteriores, optimizando la reutilización de código y minimizando el mantenimiento de scripts en el servidor.
- **Filtros de Seguridad de Catálogo:** Se incluyó una cláusula de control `ps.index_id > 0` para ignorar los *Heaps* (estructuras de tablas amontonadas sin ordenamiento físico). Esto garantiza que el motor asigne recursos únicamente a los índices reales (*Clustered* y *NonClustered*).

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso de Bloques de Control de Errores y Retorno Temprano:** Se utiliza RAISERROR acoplado con una sentencia RETURN. Esto detiene de inmediato el hilo de ejecución si hay dependencias faltantes en el esquema, previniendo fallas en cascada durante ejecuciones automáticas de medianoche en el *SQL Server Agent*.
- **Preservación del Estándar Corporativo ANSI en Mayúsculas:** Toda la sintaxis de definición relacional y manipulación del sistema se apega fielmente al estándar formal de codificación de base de datos (CREATE VIEW, DROP VIEW, SELECT, CASE...WHEN...ELSE...END), manteniendo un entorno limpio y profesional.
- **Limpieza de Tipos de Datos (Casting Explicito):** Al aplicar CAST(ps.avg_fragmentation_in_percent AS DECIMAL(5,2)), se reduce el tamaño de almacenamiento en el flujo del reporte visual, eliminando decimales excesivos e innecesarios que saturan los logs de control.

TEMA 3. Administración de transacciones y bloqueos

Proyecto 1: Control de Concurrency y Bloqueos en Matrículas

1. Enunciado del ejercicio

Durante el proceso de matrícula simultánea de estudiantes se han detectado inconsistencias en el stock de vacantes de asignaturas y registros duplicados en la base de datos **GradosTitulos**. Se solicita desarrollar un script en T-SQL que implemente una transacción robusta para el registro de una matrícula bajo los niveles de aislamiento adecuados. La solución debe:

- Controlar de forma estricta las propiedades ACID.
- Manejar bloqueos para evitar lecturas sucias (*Dirty Reads*) y actualizaciones perdidas (*Lost Updates*).
- Implementar una estructura de control de errores que garantice que, ante cualquier falla, el sistema regrese a su estado original (Rollback).

2. Script de la solución en T-SQL

```

-- =====
-- TEMA 3 - PROYECTO 1: Control de Concurrencia y Transacciones Robustas
-- Base de Datos: GradosTitulos
-- =====

USE GradosTitulos;
GO

-- -----
-- PASO 1: Configuración del Nivel de Aislamiento
-- Se utiliza REPEATABLE READ para evitar que otra transacción modifique los
-- datos leídos (vacantes) hasta que esta transacción finalice (evita Lost Updates).
-- -----
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
GO

-- -----
-- PASO 2: Procedimiento Almacenado Seguro para Registro de Matrícula
-- -----
IF EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[USP_Regist:
BEGIN
    DROP PROCEDURE [dbo].[USP_Registrar_Matricula_Segura];
END
GO

CREATE PROCEDURE [dbo].[USP_Registrar_Matricula_Segura]
    @IdPersona INT,
    @IdAsignatura INT,
    @Seccion NVARCHAR(10)
AS
BEGIN
    -- Desactivar el conteo de filas para optimizar la comunicación en red
    SET NOCOUNT ON;

    -- Declaración de variables de control interno
    DECLARE @VacantesDisponibles INT;
    DECLARE @IdMatriculaGenerada INT;

```

3. Justificación técnica de la solución aplicada

- **Nivel de Aislamiento REPEATABLE READ:** Se configuró globalmente para la sesión del procedimiento. Este nivel coloca bloqueos compartidos en todas las filas leídas por la consulta y los retiene hasta que termine la transacción, impidiendo que otros usuarios alteren el número de vacantes simultáneamente y solucionando el fenómeno de *lecturas no repetibles* o *actualizaciones perdidas*.
- **Uso Estratégico de Hints (XLOCK, ROWLOCK):** Al ejecutar la lectura del stock de cupos, se forzó un bloqueo exclusivo a nivel de fila (WITH (XLOCK, ROWLOCK)). Esto evita el problema de concurrencia clásica donde dos estudiantes leen al mismo tiempo que queda "1 vacante" y ambos avanzan al bloque de inserción. El primero en llegar duerme al segundo hilo hasta liberar la operación.
- **Preservación Estricta de la Atomicidad:** La estructura encapsulada bajo un bloque BEGIN TRANSACTION garantiza que el registro principal de matrícula, el detalle del curso y la reducción de

la vacante se comporten como una única unidad de trabajo indivisible. Si falla el paso 4 o 5, el motor ejecuta un ROLLBACK total, impidiendo datos huérfanos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso de SCOPE_IDENTITY() en Lugar de @@IDENTITY:** Para capturar la llave primaria generada del registro de matrícula, se utilizó SCOPE_IDENTITY(). Esta es la práctica recomendada de seguridad, ya que limita el alcance al módulo actual, evitando capturar por error valores generados por disparadores (*Triggers*) que actúen en segundo plano sobre otras tablas.
- **Control de Transacciones mediante @@TRANCOUNT:** Dentro del bloque CATCH, se realiza la validación condicional IF @@TRANCOUNT > 0 antes de invocar un ROLLBACK. Esto previene fallas de ejecución del motor en caso de que la transacción se haya cerrado automáticamente por un error crítico del servidor de nivel de sistema.
- **Estándar de Nombres Calificados y Palabras Clave:** Al igual que en entornos corporativos, todas las sentencias del motor SQL se escriben en mayúsculas sostenidas y se especifican de manera explícita los esquemas de bases de datos ([Academico].[Matricula]), acelerando el mapeo en el catálogo interno de SQL Server.

Proyecto 2: Resolución de Interbloqueos (Deadlocks) y Control de Prioridades

1. Enunciado del ejercicio

La Oficina de Tecnologías de Información ha detectado la presencia de *Deadlocks* (interbloqueos) cuando se procesan simultáneamente la actualización de trámites de titulación y la carga de documentos de validación académica en la base de datos **GradosTitulos**. El proceso A bloquea la tabla de trámites esperando la de documentos, mientras que el proceso B hace lo opuesto de manera simultánea.

Se solicita diseñar una estrategia técnica en T-SQL que:

- Simule controladamente un escenario de interbloqueo utilizando dos hilos de ejecución paralelos.
- Resuelva el conflicto implementando la directiva de prioridades de bloqueo (DEADLOCK_PRIORITY).
- Estructure el código de modo que el proceso elegido como víctima capture la excepción, espere un tiempo prudencial y realice un reintento automático de la transacción sin afectar la experiencia del usuario.

2. Script de la solución en T-SQL


```

-- -----
-- SOLUCIÓN TÉCNICA: Procedimiento con Manejo de Prioridad y Reintentos
-- -----

IF EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[USP_ActualizarTramiteConReintento]'))
BEGIN
    DROP PROCEDURE [dbo].[USP_ActualizarTramiteConReintento];
END
GO

CREATE PROCEDURE [dbo].[USP_ActualizarTramiteConReintento]
    @IdTramite INT,
    @NuevoEstado NVARCHAR(50),
    @EstadoArchivo NVARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;

    -- Definir que este proceso tiene prioridad alta en el servidor.
    -- Si compete con otro de prioridad normal/baja, el otro será elegido como víctima.
    SET DEADLOCK_PRIORITY HIGH;

    DECLARE @Intentos INT = 1;
    DECLARE @MaxIntentos INT = 3;
    DECLARE @TransaccionCompletada BIT = 0;

    -- Lazo de control para el manejo del reintento automático
    WHILE @Intentos <= @MaxIntentos AND @TransaccionCompletada = 0
    BEGIN
        BEGIN TRY
            BEGIN TRANSACTION;

            -- Operación 1: Modificar la tabla de Trámites
            UPDATE [Tramite].[Tramite]
            SET [EstadoTramite] = @NuevoEstado,
                [FechaModificacion] = GETDATE()
            WHERE [IdTramite] = @IdTramite;

```

3. Justificación técnica de la solución aplicada

- **Asignación de Prioridad con SET DEADLOCK_PRIORITY HIGH:** Al establecer la prioridad en HIGH (Alta), le indicamos explícitamente al motor transaccional de SQL Server que, ante una colisión circular de recursos, proteja este hilo y elija de manera mandatoria como *Deadlock Victim* al proceso concurrente que tenga una prioridad inferior (NORMAL o LOW), salvaguardando la integridad de los datos críticos de titulación.
- **Captura Específica del Error 1205:** El bloque CATCH no asume que todas las excepciones son iguales. Utiliza la función `ERROR_NUMBER() = 1205`, el identificador numérico nativo e invariable con el que SQL Server notifica que una transacción ha sido abortada para romper un interbloqueo.
- **Estrategia de Reintento con Retraso (*Backoff Delay*):** El uso del comando `WAITFOR DELAY '00:00:02'` es fundamental. Al obligar al hilo de ejecución a dormir durante 2 segundos antes de volver a entrar al ciclo WHILE, le permite al otro proceso en conflicto finalizar completamente sus operaciones y liberar sus bloqueos, eliminando la contención de hilos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Garantía Absoluta contra Bloqueos Huérfanos:** Al incluir la instrucción de control condicional `IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION` dentro del manejo de excepciones, se asegura de que, si la transacción fue cancelada a la mitad del camino, las conexiones físicas regresen al estado neutral del pool sin dejar recursos tomados.
- **Uso de Variables de Control de Estado:** En lugar de forzar salidas abruptas con `BREAK` o `GOTO`, se implementó una variable booleana de control (`@TransaccionCompleta`). Esto mantiene el flujo del código limpio, estructurado y alineado con los estándares internacionales de la ingeniería de software.
- **Formato Limpio y Uso de Esquemas:** Se mantiene la nomenclatura calificada de objetos envolviendo las tablas bajo sus respectivos esquemas de base de datos (`[Tramite].[Tramite]`), lo que optimiza el tiempo de compilación del plan de ejecución por parte del motor de SQL Server.

Proyecto 3: Niveles de Aislamiento Avanzados (Snapshot Isolation)

1. Enunciado del ejercicio

La Oficina de Registro Académico necesita generar reportes estadísticos consolidados del estado de los trámites de titulación. Sin embargo, al ejecutar estos reportes (que leen miles de registros), se bloquea temporalmente a los secretarios que intentan insertar o actualizar trámites en tiempo real.

Se solicita implementar una solución basada en **Snapshot Isolation** (Aislamiento de Instantánea) en la base de datos **GradosTítulos**. La solución debe permitir:

- Que las consultas de reportes no bloqueen las operaciones de actualización.
- Que los reportes lean una versión consistente de los datos en el tiempo (*versión de filas en TempDB*), ignorando los bloqueos por transacciones concurrentes.
- Activar las propiedades a nivel de base de datos y construir el script de prueba transaccional.

2. Script de la solución en T-SQL

```

USE master;
GO

-- -----
-- PASO 1: Habilitar el Aislamiento de Instantánea a Nivel de Base de Datos
-- Nota: Es necesario asegurar que no existan conexiones activas bloqueando
-- el cambio de estado de la base de datos.
-- -----

ALTER DATABASE GradosTitulos
SET ALLOW_SNAPSHOT_ISOLATION ON;
GO

ALTER DATABASE GradosTitulos
SET READ_COMMITTED_SNAPSHOT ON WITH ROLLBACK IMMEDIATE;
GO

USE GradosTitulos;
GO

-- -----
-- PASO 2: Procedimiento para Generar Reporte Consolidado Bajo SNAPSHOT
-- -----

IF EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[USP_Reporte_Tramites_Snapshot]'))
BEGIN
    DROP PROCEDURE [dbo].[USP_Reporte_Tramites_Snapshot];
END
GO

CREATE PROCEDURE [dbo].[USP_Reporte_Tramites_Snapshot]
AS
BEGIN
    -- Establecer explícitamente el nivel de aislamiento avanzado para esta sesión
    SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
    SET NOCOUNT ON;

    BEGIN TRY
        -- El reporte inicia una transacción de lectura consistente
        BEGIN TRANSACTION;

        -- Consulta analítica que recorre los esquemas clave sin bloquear ni ser blo
        SELECT
            t.[EstadoTramite] AS [Estado del Trámite],
            COUNT(t.[IdTramite]) AS [Total_Casos],

```

3. Justificación técnica de la solución aplicada

- **Uso de Contenedores de Versiones (*Row Versioning*):** Al activar ALLOW_SNAPSHOT_ISOLATION y READ_COMMITTED_SNAPSHOT, el motor de SQL Server utiliza la base de datos del sistema TempDB para almacenar una copia o instantánea (*snapshot*) de las filas modificadas. Cuando el reporte se ejecuta, lee la versión guardada en TempDB, eliminando la necesidad de adquirir bloqueos compartidos (*Shared Locks*) en las tablas operativas.
- **Directiva WITH ROLLBACK IMMEDIATE:** Al alterar el estado de la base de datos, esta opción desconecta de manera controlada y automática cualquier sesión concurrente que impida la migración del nivel de aislamiento, garantizando la correcta transición técnica del entorno.
- **Eliminación de Contención Escritor-Lector:** Con esta arquitectura transaccional se resuelve el problema de la OTI: los secretarios pueden escribir datos pesados (INSERT/UPDATE) sin colisionar con los procesos analíticos de auditoría o los reportes gerenciales de la universidad.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso del Hint WITH (READPAST) en Entornos Altamente Concurrentes:** Se incluyó como salvaguarda complementaria para indicarle al motor que si encuentra una página de datos bloqueada a nivel de fila por una inserción cruda que aún no se confirma, salte de manera segura ese registro en lugar de pausar el hilo de lectura analítica.
- **Declaración Explícita del Ámbito Transaccional:** La instrucción SET TRANSACTION ISOLATION LEVEL SNAPSHOT está confinada estrictamente dentro del cuerpo del procedimiento, asegurando que el cambio avanzado de concurrencia afecte únicamente al módulo de reportes sin alterar el comportamiento global por defecto del servidor.
- **Formato Estandarizado de Control de Errores:** Se aplican de manera uniforme las funciones de captura del sistema (ERROR_MESSAGE(), ERROR_SEVERITY()) manteniendo consistencia técnica estricta con los entregables anteriores del proyecto.

TEMA 4. Análisis de planes de ejecución

Proyecto 1: Auditoría de Cambios de Estado en Expedientes de Grados y Títulos

1. Enunciado del ejercicio

La Oficina de Grados y Títulos requiere un mecanismo de seguridad estricto que registre de forma obligatoria cualquier modificación en el estado de los expedientes de titulación (por ejemplo, cuando pasa de 'Pendiente' a 'Aprobado' o 'Rechazado'). Se solicita diseñar e implementar un disparador de auditoría a nivel de datos (*DML After Update Trigger*) sobre la tabla **Tramite**. **Tramite** que:

- Capture de forma automática los valores previos y nuevos del estado del trámite.
- Registre la identidad del usuario del sistema que realizó el cambio, la fecha y hora exacta, y la aplicación cliente utilizada.
- Guarde toda la información en una tabla histórica de auditoría dedicada para fines de control interno y prevención de fraude.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- -----
-- PASO 1: Creación de la Tabla de Auditoría Histórica
-- -----
IF NOT EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[Auditoria_Estados_Tramite]'))
BEGIN
    CREATE TABLE [dbo].[Auditoria_Estados_Tramite] (
        [IdAuditoria] INT IDENTITY(1,1) PRIMARY KEY,
        [IdTramite] INT NOT NULL,
        [EstadoAnterior] NVARCHAR(50),
        [EstadoNuevo] NVARCHAR(50),
        [UsuarioEjecutor] NVARCHAR(128) DEFAULT SUSER_SNAME(),
        [AplicacionCliente] NVARCHAR(128) DEFAULT APP_NAME(),
        [FechaRegistro] DATETIME DEFAULT GETDATE()
    );
END
GO

-- -----
-- PASO 2: Creación del Disparador (Trigger) de Auditoría Avanzada
-- -----
IF EXISTS (SELECT * FROM sys.triggers WHERE object_id = OBJECT_ID(N'[Tramite].[TR_Auditar_Cambio_Estado_Tramite]'))
BEGIN
    DROP TRIGGER [Tramite].[TR_Auditar_Cambio_Estado_Tramite];
END
GO

CREATE TRIGGER [Tramite].[TR_Auditar_Cambio_Estado_Tramite]
ON [Tramite].[Tramite]
AFTER UPDATE
AS
BEGIN
    -- Evitar que el trigger interfiera con operaciones masivas de datos
    SET NOCOUNT ON;

    -- Validar si la columna EstadoTramite ha sido modificada en la instrucción UPDATE
    IF UPDATE([EstadoTramite])
    BEGIN

```

3. Justificación técnica de la solución aplicada

- **Uso de la Función Condicional IF UPDATE():** Se incluyó esta directiva nativa del motor a fin de optimizar el rendimiento. El disparador evalúa instantáneamente si la columna específica EstadoTramite formó parte de la sentencia de modificación; si no fue alterada, el proceso finaliza de inmediato, evitando consumos innecesarios de CPU.
- **Cruce Analítico de las Tablas Temporales inserted y deleted:** Los disparadores en SQL Server gestionan de forma exclusiva dos tablas lógicas residentes en memoria caché. Se utilizó deleted para extraer la versión original del registro (el estado anterior) e inserted para capturar la nueva actualización, realizando un acoplamiento por clave primaria (IdTramite).
- **Captura de Metadatos del Servidor:** Al utilizar las funciones del sistema SUSER_SNAME() y APP_NAME(), se extraen las identidades de red reales de los usuarios y el software desde donde se conectan (por ejemplo, .NET Web Application o SQL Server Management Studio), garantizando el principio de no repudio de la información.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Filtrado contra Actualizaciones Redundantes:** El predicado `WHERE i.[EstadoTramite] <> d.[EstadoTramite]` es una regla clave de optimización. Previene que se guarden filas en la tabla de auditoría si un usuario ejecuta un comando que sobrescribe exactamente el mismo estado que el registro ya poseía.
- **Uso Defensivo de Esquemas en Objetos y Triggers:** Al definir el disparador se especificó de manera calificada el esquema correspondiente (`ON [Tramite].[Tramite]`), manteniendo la estructura modular y protegiendo el catálogo de datos de la universidad.
- **Control de Idempotencia Estricto:** Al igual que en las fases previas, se incorporan sentencias condicionales de validación física de objetos (`IF EXISTS / IF NOT EXISTS`) para asegurar el despliegue del script de forma limpia en cualquier ambiente de base de datos.

Proyecto 2: Triggers Preventivos (INSTEAD OF TRIGGERS) para Reglas de Negocio Complejas

1. Enunciado del ejercicio

La normativa institucional de la universidad estipula de forma estricta que no se puede eliminar ningún registro de expediente de trámite de titulación de la tabla **Tramite.Tramite** si este ya se encuentra en estado '**Aprobado**' o '**En Proceso**'. Además, por cuestiones de integridad referencial histórica, las eliminaciones de trámites en estado '**Pendiente**' o '**Rechazado**' no deben borrar físicamente las filas de la base de datos, sino que deben realizar un **borrado lógico** modificando el estado del trámite a '**Eliminado**'.

Se solicita diseñar e implementar un disparador preventivo **INSTEAD OF DELETE** para interceptar de manera segura los comandos de eliminación y forzar el cumplimiento automático de estas reglas de negocio.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- -----
-- PASO 1: Creación del Disparador Preventivo INSTEAD OF
-- -----

IF EXISTS (SELECT * FROM sys.triggers WHERE object_id = OBJECT_ID(N'[Tramite].[TR_Prevenir_Eliminacion_Fisica_Tramite]'))
BEGIN
    DROP TRIGGER [Tramite].[TR_Prevenir_Eliminacion_Fisica_Tramite];
END
GO

CREATE TRIGGER [Tramite].[TR_Prevenir_Eliminacion_Fisica_Tramite]
ON [Tramite].[Tramite]
INSTEAD OF DELETE
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Regla de Negocio: Bloquear y cancelar la eliminación si hay registros protegidos
    IF EXISTS (
        SELECT 1
        FROM deleted
        WHERE [EstadoTramite] IN ('Aprobado', 'En Proceso', 'Aprobado Por Consejo')
    )
    BEGIN
        RAISERROR('Operación Rechazada: No se permite eliminar expedientes que ya están en proceso', 16, 1);
        ROLLBACK TRANSACTION; -- Revierte inmediatamente el comando DML que gatilló el trigger
        RETURN;
    END

    -- 2. Regla de Negocio: Aplicar Borrado Lógico para los estados permitidos ('Pendiente', 'Rechazado')
    UPDATE t
    SET t.[EstadoTramite] = 'Eliminado',
        t.[FechaModificacion] = GETDATE()
    FROM [Tramite].[Tramite] t
    INNER JOIN deleted d ON t.[IdTramite] = d.[IdTramite]
    WHERE d.[EstadoTramite] IN ('Pendiente', 'Rechazado');

    PRINT 'Borrado físico cancelado. Se aplicó borrado lógico exitosamente sobre los registros protegidos.';
END
GO

```

3. Justificación técnica de la solución aplicada

- **Arquitectura del Disparador INSTEAD OF:** A diferencia de un trigger AFTER (que actúa después de que el cambio impactó las páginas de datos), un disparador INSTEAD OF sustituye por completo la acción del comando DELETE. El motor transaccional de SQL Server no llega a tocar la tabla base físicamente; en su lugar, ejecuta las directivas de programación de control escritas dentro del bloque del trigger.
- **Uso Decisivo de ROLLBACK TRANSACTION:** Si la validación lógica detecta un registro protegido por la universidad a través de la tabla virtual de memoria deleted, el trigger levanta una excepción controlada mediante RAISERROR y ejecuta un ROLLBACK, cancelando todo el lote (*batch*) de ejecución de manera segura.
- **Mapeo Orientado a Conjuntos en Borrado Lógico:** La instrucción UPDATE del disparador realiza un cruce (*JOIN*) directo con la tabla lógica deleted. Esto asegura que el trigger se comporte de forma

óptima tanto para la eliminación de una sola fila como para eliminaciones masivas por lotes de expedientes.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Prevención de Operaciones Huérfanas con RETURN:** Colocar la instrucción RETURN inmediatamente después del ROLLBACK asegura la salida limpia del hilo de ejecución del disparador, evitando que el motor intente procesar las siguientes líneas de código del trigger de forma errónea.
- **Preservación de la Integridad Referencial:** Al sustituir la eliminación física por una actualización de estado a 'Eliminado', se previene la ruptura de llaves foráneas (*Foreign Keys*) en cascada con las tablas hijas de auditoría, documentos y requisitos de titulación.
- **Garantía de Uniformidad en Código Limpio:** El script utiliza de manera estricta mayúsculas fijas para las palabras clave del sistema y encierra adecuadamente los objetos del motor dentro de su correspondiente esquema relacional ([Tramite].[Tramite]).

Proyecto 3: Seguridad de Estructuras mediante Triggers DDL (Data Definition Language)

1. Enunciado del ejercicio

Para mitigar riesgos operativos o alteraciones no autorizadas por parte del personal técnico, la Oficina de Tecnologías de Información de la universidad exige proteger la arquitectura relacional de la base de datos GradosTitulos.

Se solicita implementar un disparador de seguridad a nivel de Definición de Datos (**DDL Trigger**) que:

- Impida la eliminación (DROP) o modificación estructural (ALTER) de cualquier tabla o vista de los esquemas del sistema.
- Capture la información técnica del evento mediante el uso de funciones XML integradas (EVENTDATA()).
- Guarde un rastro detallado del incidente (sentencia ejecutada, fecha, usuario y terminal) en una tabla centralizada de control de cambios.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- -----
-- PASO 1: Creación de la Tabla de Log de Cambios Estructurales
-- -----
IF NOT EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[Log_Ca
BEGIN
    CREATE TABLE [dbo].[Log_Cambios_Estructurales_DDL] (
        [IdLogDDL] INT IDENTITY(1,1) PRIMARY KEY,
        [FechaEvento] DATETIME DEFAULT GETDATE(),
        [TipoEvento] NVARCHAR(100),
        [NombreObjeto] NVARCHAR(256),
        [TipoObjeto] NVARCHAR(100),
        [UsuarioDB] NVARCHAR(128),
        [ComandoEjecutado] NVARCHAR(MAX),
        [DatosXMLCompleto] XML
    );
END
GO

-- -----
-- PASO 2: Creación del Disparador DDL de Bloqueo y Auditoría
-- -----
IF EXISTS (SELECT * FROM sys.triggers WHERE name = 'TR_Proteger_Arquitectura_BaseData
BEGIN
    DROP TRIGGER TR_Proteger_Arquitectura_BaseDatos ON DATABASE;
END
GO

CREATE TRIGGER [TR_Proteger_Arquitectura_BaseDatos]
ON DATABASE
FOR ALTER_TABLE, DROP_TABLE, ALTER_VIEW, DROP_VIEW
AS
BEGIN
    SET NOCOUNT ON;

    -- Declarar variable para capturar la estructura de metadatos del evento
    DECLARE @EventData XML;
    SET @EventData = EVENTDATA();

```

3. Justificación técnica de la solución aplicada

- **Ámbito de Ejecución ON DATABASE:** A diferencia de los disparadores tradicionales (DML) acoplados directamente a las filas de una tabla, el trigger DDL se define bajo el contexto de la base de datos completa. Esto le confiere la capacidad de interceptar cualquier comando del diccionario de datos en el servidor.
- **Procesamiento de Estructuras mediante la Función EVENTDATA():** Cuando ocurre un evento DDL, SQL Server genera internamente un fragmento de datos en formato XML con toda la metadata del hilo de ejecución. Se utilizó la asignación a una variable de tipo XML acoplada con métodos de extracción jerárquica .value() para estructurar el reporte de incidentes en columnas de datos relacionales sin degradar el rendimiento.
- **Uso imperativo de ROLLBACK TRANSACTION:** Al ejecutarse un comando DROP o ALTER, el disparador intercepta la transacción del catálogo del motor de base de datos. Al aplicar un

ROLLBACK, se cancela de forma inmediata la actualización de los metadatos de las tablas del sistema, manteniendo la base de datos intacta.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso del Bloque Condicional `parent_class = 0`:** En la sección de validación previa de existencia, se especificó la propiedad de clase de objeto correspondiente a la base de datos. De esta forma se previene confusiones sintácticas del parser con triggers tradicionales a nivel de tabla.
- **Almacenamiento de Respaldo XML (`DatosXMLCompleto`):** Además de guardar la información en columnas legibles individuales, se almacenó de manera íntegra la cadena nativa de `EVENTDATA()` en la base de datos. Esto asegura contar con evidencia digital forense completa del entorno técnico del servidor ante una auditoría externa de TI.
- **Nomenclatura Limpia y en Mayúsculas Sostenidas:** Se mantiene la adherencia absoluta al estándar de diseño limpio de SQL para palabras claves corporativas, facilitando que el código sea autodocumentado y escalable.

TEMA 5. OPTIMIZACIÓN DE CONSULTAS T-SQL

Proyecto 1: Reescritura de consultas de alta demanda

1. Enunciado del ejercicio

Las consultas utilizadas para mostrar el historial completo del trámite de un estudiante consumen excesivos recursos. Se solicita optimizar dichas consultas mediante:

- Eliminación de subconsultas innecesarias.
- Uso de JOIN eficientes.
- Uso de EXISTS.
- Uso de CTE.
- Comparación de tiempos.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- Activar estadísticas de tiempo y asignación de páginas de memoria para la comparación
SET STATISTICS TIME ON;
SET STATISTICS IO ON;
GO

-- -----
-- CÓDIGO ANTERIOR: Consulta Ineficiente (Uso de subconsultas en SELECT e IN)
-- -----

PRINT '=== EJECUCIÓN DE LA CONSULTA INEFICIENTE (ANTERIOR) ===';
SELECT
    t.[IdTramite],
    t.[EstadoTramite],
    (SELECT TOP 1 p.[Nombre] FROM [Academico].[Persona] p WHERE p.[IdPersona] = t.[IdPersona]) AS [EstudianteNombre],
    t.[FechaModificacion]
FROM [Tramite].[Tramite] t
WHERE t.[IdTramite] IN (
    SELECT r.[IdTramite]
    FROM [Documento].[Requisito] r
    WHERE r.[EstadoArchivo] = 'Validado'
);
GO

-- -----
-- CÓDIGO OPTIMIZADO: Implementación de CTE, JOINS eficientes y EXISTS
-- -----

PRINT '=== EJECUCIÓN DE LA CONSULTA REESCRITA Y OPTIMIZADA ===';
WITH CTE_Estudiantes AS (
    SELECT
        [IdPersona],
        [Nombre] AS [EstudianteNombre]
    FROM [Academico].[Persona]
)
SELECT
    t.[IdTramite],
    t.[EstadoTramite],
    e.[EstudianteNombre] AS [Estudiante],
    t.[FechaModificacion]
FROM [Tramite].[Tramite] t

```

3. Justificación técnica de la solución aplicable

- **Sustitución de Subconsultas Correlacionadas por JOINS:** La consulta original ejecutaba una subconsulta en la lista de selección SELECT (SELECT TOP 1 . . .) por cada una de las filas devueltas por la consulta externa (*Efecto RBAR: Row-By-Agonizing-Row*). Al encapsular la tabla mediante una Expresión de Tabla Común (CTE) y acoplarla con un INNER JOIN, el optimizador de SQL Server combina los conjuntos en memoria en un solo paso utilizando operadores óptimos como *Merge Join* o *Hash Match*.
- **Predicado de Cortocircuito con EXISTS:** Reemplazar el operador subóptimo IN por la cláusula condicional EXISTS cambia radicalmente el plan físico. IN obliga al motor a evaluar y construir toda la lista de valores de la subconsulta antes de filtrar. En cambio, EXISTS detiene la búsqueda en las páginas del índice inmediatamente después de encontrar la primera coincidencia satisfactoria para la fila evaluada (*Short-circuit evaluation*).

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Monitoreo Técnico Mediante Comandos de Sesión:** El uso explícito de SET STATISTICS TIME ON y SET STATISTICS IO ON es la mejor práctica estándar para cuantificar la optimización. Permite auditar el ahorro exacto en milisegundos de tiempo de CPU y la drástica reducción de lecturas lógicas (páginas de datos cargadas desde la memoria caché).
- **Modularidad con CTE (Common Table Expressions):** Las CTEs no alteran el rendimiento por sí solas, pero estructuran el código de forma limpia, facilitando al motor la reutilización de expresiones comunes y eliminando anidamientos complejos de subconsultas difíciles de auditar.

Proyecto 2: Optimización de reportes académicos

1. Enunciado del ejercicio

Los reportes institucionales integran información proveniente de más de ocho tablas. Se solicita desarrollar una versión optimizada utilizando:

- CTE.
- Funciones ventana.
- Índices adecuados.
- Filtrado temprano.
- Estadísticas actualizadas.
- Comparar rendimiento antes y después.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- -----
-- PASO 1: Filtrado Temprano y Estructuración mediante CTE y Funciones Ventana
-- -----

PRINT '=== EJECUCIÓN DEL REPORTE ACADÉMICO OPTIMIZADO ===';
SET STATISTICS TIME ON;
SET STATISTICS IO ON;
GO

-- Definición de CTEs para aislar y filtrar los conjuntos de datos de forma temprana
WITH CTE_Persona_Filtrada AS (
    SELECT [IdPersona], [Nombre], [DNI]
    FROM [Academico].[Persona]
    WHERE [Estado] = 'Activo' -- Filtrado Temprano: Reduce el universo de filas antes
),
CTE_Matricula_Reciente AS (
    SELECT
        [IdMatricula],
        [IdPersona],
        [FechaMatricula],
        -- Función Ventana para identificar la última matrícula sin agrupar físicamente
        ROW_NUMBER() OVER (PARTITION BY [IdPersona] ORDER BY [FechaMatricula] DESC) AS [RankMatr]
    FROM [Academico].[Matricula]
),
CTE_Tramite_Consolidado AS (
    SELECT
        [IdTramite], [IdPersona], [EstadoTramite], [TipoTramite], [FechaModificacion]
    FROM [Tramite].[Tramite]
    WHERE [EstadoTramite] NOT IN ('Rechazado', 'Eliminado') -- Filtrado Temprano
)
SELECT
    p.[DNI],
    p.[Nombre] AS [Estudiante],
    m.[FechaMatricula] AS [Última_Matricula],
    t.[TipoTramite] AS [Modalidad],
    t.[EstadoTramite] AS [Estado_Expediente],
    -- Función ventana analítica sobre el total de trámites del alumno
    COUNT(t.[IdTramite]) OVER(PARTITION BY p.[IdPersona]) AS [Total_Tramites_Alumno]
FROM CTE_Persona_Filtrada p
INNER JOIN CTE_Matricula_Reciente m ON p.[IdPersona] = m.[IdPersona] AND m.[RankMatr] = 1
INNER JOIN CTE_Tramite_Consolidado t ON p.[IdPersona] = t.[IdPersona]

```

3. Justificación técnica de la solución aplicada

- **Beneficio del Filtrado Temprano (*Filter Pushdown*):** En lugar de realizar los JOINS de las 8 tablas con toda la data cruda y luego aplicar un WHERE global, las CTEs ejecutan filtros lógicos internos (WHERE [Estado] = 'Activo'). Esto reduce drásticamente el volumen de registros en los operadores internos del plan de ejecución antes de procesar los acoplamientos físicos.
- **Funciones Ventana (ROW_NUMBER y COUNT OVER) sin Bloqueos:** El uso de funciones ventana evita la necesidad de recurrir a la cláusula tradicional GROUP BY combinada con subconsultas repetitivas. Al calcular los agregados sobre particiones lógicas en memoria (OVER(PARTITION BY...)), el motor realiza un único escaneo (*Scan*) del conjunto de datos optimizando el coste de CPU.
- **Muestreo Exacto por FULLSCAN:** Forzar la actualización con WITH FULLSCAN obliga a SQL Server a examinar el 100% de las filas de las tablas académicas en lugar de usar un muestreo porcentual por

defecto. Esto garantiza que el optimizador estime con absoluta precisión la selectividad de los datos, eligiendo el plan de ejecución más veloz.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Índices de Cobertura (*Covering Indexes*):** Al diseñar el índice `IX_Matricula_IdPersona_Fecha` usando la cláusula `INCLUDE ([IdMatricula])`, se colocan los datos de la clave primaria directamente en las páginas hoja del índice secundario. Esto evita que el optimizador tenga que saltar a la tabla base a buscar la clave primaria, eliminando la costosa operación de búsqueda de clave (*Key Lookup*).
- **Insolución del RBAR:** Al omitir el uso de bucles o funciones escalares personalizadas dentro del select del reporte, el script se adhiere al paradigma orientado a conjuntos de bases de datos relacionales, garantizando un rendimiento escalable a lo largo del tiempo.

Proyecto 3: Optimización de procedimientos almacenados

1. Enunciado del ejercicio

Los procedimientos almacenados principales que controlan la lógica de graduación de alumnos han empezado a degradarse y demorarse debido al fenómeno de *Parameter Sniffing* o planes de ejecución obsoletos. Se solicita reestructurar el componente analítico central implementando técnicas avanzadas de optimización de código compilado que incluyan:

- Uso de variables locales.
- Uso de HINTS de optimización (`RECOMPILE` u `OPTIMIZE FOR`).
- Evitar funciones escalares en cláusulas `WHERE`.
- Identificar y remover bloqueos innecesarios.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

IF EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[USP_Consultar_Expdientes_Graduacion_Optimizado]'))
BEGIN
    DROP PROCEDURE [dbo].[USP_Consultar_Expdientes_Graduacion_Optimizado];
END
GO

CREATE PROCEDURE [dbo].[USP_Consultar_Expdientes_Graduacion_Optimizado]
    @IdPersona INT,
    @EstadoTramite NVARCHAR(50)
AS
BEGIN
    -- Desactivar el conteo de filas e indicar aislamiento de lectura no bloqueante
    SET NOCOUNT ON;

    -- 1. Solución al Parameter Sniffing: Declaración de variables locales
    -- El motor compilará un plan basado en estadísticas promedio y no en el valor de los parámetros
    DECLARE @IdPersona_Local INT = @IdPersona;
    DECLARE @EstadoTramite_Local NVARCHAR(50) = @EstadoTramite;

    BEGIN TRY
        -- 2. Consulta principal optimizada y protegida contra bloqueos innecesarios
        SELECT
            t.[IdTramite],
            t.[IdPersona],
            t.[TipoTramite],
            t.[EstadoTramite],
            t.[FechaModificacion]
        FROM [Tramite].[Tramite] t WITH (NOLOCK) -- Remueve bloqueos compartidos de lectura
        WHERE t.[IdPersona] = @IdPersona_Local
            AND t.[EstadoTramite] = @EstadoTramite_Local
        -- Buena práctica: Se evita usar funciones del sistema (como LEFT o UPPER)
        -- para permitir búsquedas eficientes por rango (Index Seek).

        -- 3. Uso del Hint de Optimización para forzar un plan equilibrado
        OPTION (OPTIMIZE FOR (@IdPersona_Local UNKNOWN, @EstadoTramite_Local = 'Aprobado'));

    END TRY
    BEGIN CATCH
        DECLARE @ErrMsg NVARCHAR(4000) = ERROR_MESSAGE();
    END CATCH
END

```

3. Justificación técnica de la solución aplicada

- **Mitigación del *Parameter Sniffing*:** Cuando un procedimiento se ejecuta por primera vez, SQL Server husmea (*sniffs*) los parámetros de entrada para compilar un plan adaptado a ese valor específico. Si el primer alumno consultado tiene 1 trámite, pero el segundo tiene 10,000, el plan original provocará un cuello de botella catastrófico. Copiar los parámetros a variables locales u usar `OPTIMIZE FOR (... UNKNOWN)` obliga al optimizador a basarse en la densidad general de las estadísticas, generando un plan genérico estable.
- **Eliminación de Expresiones no Sgargables (*Non-Sargable*):** Al retirar funciones escalares o transformaciones de caracteres (como `UPPER(t.EstadoTramite) = ...`) directamente aplicadas sobre las columnas en el predicado `WHERE`, se le devuelve al motor la capacidad de efectuar búsquedas directas por puntero (*Index Seek*) sobre los índices existentes en lugar de recorrer toda la tabla físicamente (*Index Scan*).
- **Supresión de Bloqueos mediante `WITH (NOLOCK)`:** En procedimientos que generan data informativa masiva para graduaciones, adjuntar el modificador de tabla `NOLOCK` instruye al gestor transaccional a omitir la adquisición de bloqueos compartidos (*Shared Locks*), evitando bloqueos y esperas de hilo con los operarios que registran documentos en vivo.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso de la Directiva OPTION (OPTIMIZE FOR):** Actúa como una directiva explícita de compilación. Permite indicarle al planificador del motor cuál es el valor de negocio más frecuente o crítico en producción, maximizando los aciertos en la memoria caché de consultas (*Plan Cache*).
- **Estructura Defensiva Limpia:** La encapsulación estricta en bloques TRY-CATCH junto a la instrucción de red SET NOCOUNT ON minimiza la sobrecarga de paquetes en el canal de comunicación cliente-servidor de la universidad.

TEMA 6. CONTROL DE RECURSOS CON RESOURCE GOVERNOR

Proyecto 1: Administración de carga por tipo de usuario

1. Enunciado del ejercicio

La universidad desea limitar el consumo de recursos de usuarios administrativos sin afectar las consultas institucionales críticas. Se solicita configurar **Resource Governor** para:

- Crear Resource Pool.
- Crear Workload Group.
- Clasificar conexiones.
- Limitar CPU.
- Limitar memoria.
- Verificar el funcionamiento mediante pruebas.

2. Script de la solución en T-SQL

```

USE master;
GO

-- -----
-- PASO 1: Crear el Resource Pool (Contenedor de Recursos Físicos)
-- Con esto limitamos los topes de CPU y memoria asignados a los administrativos
-- -----
IF EXISTS (SELECT * FROM sys.resource_governor_resource_pools WHERE name = 'Pool_Administrativo')
BEGIN
    DROP WORKLOAD GROUP Group_Administrativo; -- Requiere limpiar dependencias primero
    DROP RESOURCE POOL Pool_Administrativo;
END
GO

CREATE RESOURCE POOL Pool_Administrativo
WITH (
    MIN_CPU_PERCENT = 0,
    MAX_CPU_PERCENT = 30,          -- Límite estricto de CPU al 30%
    MIN_MEMORY_PERCENT = 0,
    MAX_MEMORY_PERCENT = 20       -- Límite estricto de memoria RAM al 20%
);
GO

-- -----
-- PASO 2: Crear el Workload Group (Grupo de Carga de Trabajo)
-- -----
CREATE WORKLOAD GROUP Group_Administrativo
USING Pool_Administrativo; -- Vincula el grupo al contenedor limitado creado arriba
GO

-- -----
-- PASO 3: Crear la Función de Clasificación (Classifier Function)
-- -----
IF OBJECT_ID('dbo.UFN_Clasificador_Conexiones', 'FN') IS NOT NULL
BEGIN
    -- Desvincular temporalmente del gobernador para poder reconstruirla de forma limpia
    ALTER RESOURCE GOVERNOR WITH (CLASSIFIER_FUNCTION = NULL);
    ALTER RESOURCE GOVERNOR RECONFIGURE;
    DROP FUNCTION dbo.UFN_Clasificador_Conexiones;
END
GO

```

3. Justificación técnica de la solución aplicada

- **Confinamiento de Procesos con MAX_CPU_PERCENT:** Resource Governor intercepta las peticiones que ingresan al motor de almacenamiento. Al fijar el techo en 30, los hilos pertenecientes a Group_Administrativo solo obtendrán ciclos del programador (*Scheduler*) hasta ese tope si existe contención, impidiendo que una consulta pesada paralice la base de datos de los alumnos.
- **Control de Asignación por SCHEMABINDING:** La función clasificadora se compila de forma obligatoria usando esta directiva para anclar los metadatos. Esto garantiza que ningún administrador borre por error las tablas o esquemas de seguridad de donde se alimenta la función evaluadora en tiempo de conexión.
- **Uso imperativo de RECONFIGURE:** Los cambios en los objetos del gobernador se preparan en un estado diferido en memoria. El comando RECONFIGURE obliga al motor a reconstruir los metadatos internos en caliente, sin requerir el reinicio físico del servicio SQL Server de la universidad.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso Correcto de las Dependencias en Cascada:** Al limpiar entornos previos se ejecuta el script siguiendo la precedencia lógica correcta: se rompe la clasificación, se elimina el grupo de carga y por último se destruye el pool de recursos. Esto evita que el motor arroje excepciones de clave externa relacional.
- **Estrategia Fallback en Función Clasificadora:** Al incluir la cláusula ELSE SET @NombreGrupo = 'default', se asegura que si el usuario no cumple los criterios de la OTI para personal administrativo, no sea desconectado abruptamente, sino que continúe operando con los recursos base del servidor.

Proyecto 2: Priorización de procesos críticos

1. Enunciado del ejercicio

Durante el cierre del semestre, los procesos de emisión de diplomas deben tener prioridad sobre las consultas de usuarios generales. Se solicita desarrollar una solución que:

- Cree grupos de carga diferenciados.
- Priorice procesos críticos.
- Restrinja consultas de baja prioridad.
- Evalúe el impacto sobre el rendimiento.

2. Script de la solución en T-SQL

```

USE master;
GO

-- -----
-- PASO 1: Creación de Grupos de Carga Diferenciados en el Pool Principal
-- -----

-- 1. Grupo para Emisión de Diplomas (Máxima importancia institucional)
CREATE WORKLOAD GROUP Group_Diplomas_Critico
WITH (
    IMPORTANCE = HIGH,           -- Prioridad alta en la cola del planificador
    REQUEST_MAX_CPU_TIME_SEC = 0, -- Sin límite de tiempo por consulta (Proceso )
    MAX_DOP = 4                  -- Permitir paralelismo moderado para acelerar
)
USING [default];
GO

-- 2. Grupo para Consultas Generales (Baja prioridad / Restringido)
CREATE WORKLOAD GROUP Group_Usuarios_Generales
WITH (
    IMPORTANCE = LOW,           -- Prioridad baja ante contención de hilos
    REQUEST_MAX_CPU_TIME_SEC = 45, -- Restringir consultas pesadas: abortar si pa
    MAX_DOP = 1                 -- Forzar ejecución serial para mitigar la sati
)
USING [default];
GO

-- -----
-- PASO 2: Actualización de la Función Clasificadora
-- -----

ALTER RESOURCE GOVERNOR WITH (CLASSIFIER_FUNCTION = NULL);
ALTER RESOURCE GOVERNOR RECONFIGURE;
GO

ALTER FUNCTION dbo.UFN_Clasificador_Conexiones()
RETURNS SYSNAME
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @NombreGrupo SYSNAME;

```

3. Justificación técnica de la solución aplicada

- **Importancia Relativa mediante IMPORTANCE:** La propiedad IMPORTANCE (HIGH/LOW) no bloquea de forma absoluta el acceso a la CPU; en su lugar, altera la asignación de porciones de tiempo en los hilos del programador nativo de SQL Server (*SOS Scheduler*). Ante un escenario de alta concurrencia por el cierre de semestre, las peticiones del módulo de diplomas recibirán tres veces más prioridad en los ciclos de cómputo que las consultas generales de alumnos.
- **Control Estricto de Tiempos con REQUEST_MAX_CPU_TIME_SEC:** Al definir un umbral de 45 segundos para el grupo de baja prioridad, se implementa una barrera activa que destruye de forma segura los hilos de consultas mal optimizadas enviadas por usuarios generales, liberando la memoria caché y protegiendo el desempeño global.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Control del Grado Máximo de Paralelismo (MAX_DOP):** Restringir el paralelismo ($\text{MAX_DOP} = 1$) para consultas ordinarias evita que un reporte de usuario general acapare múltiples núcleos de la CPU del servidor. Al mismo tiempo, otorgar $\text{MAX_DOP} = 4$ al proceso analítico de diplomas asegura que los reportes masivos de egresados se procesen a la máxima velocidad permitida por la infraestructura relacional.
- **Mantenimiento en Caliente Libre de Interrupciones:** El flujo de desvinculación y vinculación de la función clasificadora se ejecuta usando las transiciones oficiales del motor, lo que asegura que las sesiones ya activas sigan operando normalmente sin causar caídas del servicio en la universidad.

Proyecto 3: Monitoreo del consumo de recursos

1. Enunciado del ejercicio

La Oficina de Tecnologías de la Información requiere evaluar de forma constante el impacto de las políticas de control implementadas en el Gobernador. Se solicita diseñar e implementar una solución integral de auditoría en T-SQL que permita:

- Monitorear el consumo de CPU y memoria por cada grupo de carga.
- Identificar las sesiones activas que están siendo reguladas en tiempo real.
- Almacenar periódicamente estas métricas en una tabla histórica para auditorías de desempeño y toma de decisiones en la universidad.

2. Script de la solución en T-SQL

```

USE GradosTitulos;
GO

-- -----
-- PASO 1: Creación de la Tabla Histórica de Métricas de Control
-- -----
IF NOT EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[Auditoria_Resource_Governor]'))
BEGIN
    CREATE TABLE [dbo].[Auditoria_Resource_Governor] (
        [IdAuditoriaRG] INT IDENTITY(1,1) PRIMARY KEY,
        [FechaMuestreo] DATETIME DEFAULT GETDATE(),
        [GrupoCarga] NVARCHAR(128),
        [PoolAsociado] NVARCHAR(128),
        [PeticionesActivas] INT,
        [Uso_CPU_ms] BIGINT,
        [Consumo_Memoria_KB] BIGINT,
        [Consultas_Canceladas_Timeout] BIGINT
    );
END
GO

-- -----
-- PASO 2: Procedimiento de Captura y Auditoría Periódica
-- -----
IF EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[USP_Moni_Capturar_Metricas_RG]'))
BEGIN
    DROP PROCEDURE [dbo].[USP_Moni_Capturar_Metricas_RG];
END
GO

CREATE PROCEDURE [dbo].[USP_Moni_Capturar_Metricas_RG]
AS
BEGIN
    SET NOCOUNT ON;

    -- Insertar una instantánea del estado de los contadores de Resource Governor
    INSERT INTO [dbo].[Auditoria_Resource_Governor] (
        [GrupoCarga],
        [PoolAsociado],

```

3. Justificación técnica de la solución aplicada

- **Uso de Vistas de Administración Dinámica (DMVs):** El script aprovecha `sys.dm_resource_governor_workload_groups` y `sys.dm_resource_governor_resource_pools`. Estas DMVs exponen directamente los contadores acumulativos de rendimiento en memoria del motor de SQL Server, lo que permite auditar el consumo exacto de CPU y memoria sin generar sobrecarga de disco.
- **Mapeo de Relaciones de Sesión mediante `group_id`:** Al cruzar la tabla interna de conexiones del sistema `sys.dm_exec_sessions` con el catálogo del Gobernador por medio de la columna indexada `group_id`, se logra transparencia total para identificar con precisión quirúrgica qué usuario de red o qué aplicación cliente está siendo confinada en un pool limitado en tiempo real.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Preservación del Modelo Relacional Orientado a Históricos:** Al encapsular la carga de datos dentro del procedimiento almacenado USP_Moni_Capturar_Metricas_RG, este componente queda perfectamente preparado para ser programado dentro de un Job regular del *SQL Server Agent*, ejecutándose cada hora de forma automatizada.
- **Estandarización y Formato Limpio de Datos:** Se transforman los valores nativos de páginas de memoria a kilobytes reales (`total_memgrant_count * 8`), garantizando que los reportes de control extraídos por la OTI posean unidades legibles y claras para auditorías de infraestructura.